

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
er_scaling_palier.py - Scaling ER-Blob par paliers avec checkpoint
Rudolphe Mirante - Avril 2026

Usage :
    python3 er_scaling_palier.py 1    # N = 300, 400, 500
    python3 er_scaling_palier.py 2    # N = 600, 800, 1000
    python3 er_scaling_palier.py 3    # N = 1200, 1500, 2000

Checkpoint : résultats sauvegardés dans er_scaling_results.json après chaque N.
              Reprise automatique si interrompu.
"""

import sys
import json
import time
import numpy as np
from scipy.spatial import distance_matrix

CHECKPOINT = "/mnt/user-data/outputs/er_scaling_results.json"
N_SEEDS    = 10
KAPPA0     = 8
BETA       = 0.30
TAU        = 0.50
SYM_THRESH = 0.40    # seuil de brisure (plus strict : 0.40 vs 0.50 avant)

PALIERS = {
    1: ([300, 400, 500],      {300: 3000, 400: 4000, 500: 5000}),
    2: ([600, 800, 1000],     {600: 6000, 800: 8000, 1000: 10000}),
    3: ([1200, 1500, 2000],   {1200: 12000, 1500: 15000, 2000: 20000}),
}

# -----
# Structure de graphe sparse pour MCMC rapide
# -----

class SparseBlob:
    """Graphe Blob avec adjacency-sets pour MCMC  $O(\kappa)$  par étape."""

    def __init__(self, N, adj_matrix):
        self.N          = N
        self.N_half     = N // 2

```

```

self.adj      = [set() for _ in range(N)]
self.edges    = []          # liste pour sampling rapide
self.edge_set = set()      # pour lookup O(1)
self.deg      = np.zeros(N, dtype=int)
self.n_tri    = 0

for i in range(N):
    for j in range(i+1, N):
        if adj_matrix[i, j]:
            self._add(i, j, update_tri=False)

# Compter les triangles une seule fois
self.n_tri = 0
for u, v in self.edge_set:
    self.n_tri += len(self.adj[u] & self.adj[v])
self.n_tri //= 3    # chaque triangle compté 3 fois

def _add(self, u, v, update_tri=True):
    if update_tri:
        self.n_tri += len(self.adj[u] & self.adj[v])
    self.adj[u].add(v); self.adj[v].add(u)
    key = (min(u,v), max(u,v))
    self.edges.append(key)
    self.edge_set.add(key)
    self.deg[u] += 1; self.deg[v] += 1

def _remove(self, u, v):
    self.n_tri -= len((self.adj[u] & self.adj[v]) - {u, v})
    self.adj[u].discard(v); self.adj[v].discard(u)
    key = (min(u,v), max(u,v))
    self.edge_set.discard(key)
    # Retirer de la liste (swap avec dernier)
    for idx in range(len(self.edges)-1, -1, -1):
        if self.edges[idx] == key:
            self.edges[idx] = self.edges[-1]
            self.edges.pop()
            break
    self.deg[u] -= 1; self.deg[v] -= 1

def delta_tri_add(self, u, v):
    return len(self.adj[u] & self.adj[v])

def delta_tri_remove(self, u, v):
    return len((self.adj[u] & self.adj[v]) - {u, v})

def D(self):
    mu = float(self.deg.mean())

```

```

        if mu < 1e-9: return 0.0
        return float(self.deg.std()) / mu

def Ssim(self, beta=BETA):
    return np.log(1 + self.n_tri) - beta * self.D()

def sym_deviation(self):
    """Fraction d'arêtes internes asymétriques entre A et B."""
    edges_A = {(u, v) for u, v in self.edge_set if u < self.N_half and v < se
    # Arêtes de B remappées sur A
    edges_B_mapped = {(u - self.N_half, v - self.N_half)
                      for u, v in self.edge_set
                      if u >= self.N_half and v >= self.N_half}
    union = edges_A | edges_B_mapped
    if not union: return 0.0
    sym_diff = edges_A ^ edges_B_mapped
    return len(sym_diff) / len(union)

def is_still_connected(self, u, v):
    """BFS rapide depuis u pour atteindre v (après avoir retiré l'arête u-v).
    visited = {u}
    stack = list(self.adj[u])
    while stack:
        node = stack.pop()
        if node == v:
            return True
        if node not in visited:
            visited.add(node)
            stack.extend(self.adj[node] - visited)
    return False

def random_edge(self, rng):
    idx = rng.integers(len(self.edges))
    return self.edges[idx]

def random_non_edge(self, rng, tries=200):
    for _ in range(tries):
        u = int(rng.integers(self.N))
        v = int(rng.integers(self.N))
        if u != v and v not in self.adj[u]:
            return min(u,v), max(u,v)
    return None

# -----
# Construction configuration miroir
# -----

```

```

def build_mirror(N, kappa0=KAPPA0, seed=42):
    rng = np.random.default_rng(seed)
    N_half = N // 2
    pts_A = rng.uniform(0.0, 0.45, (N_half, 3))
    pts_B = 1.0 - pts_A
    pts = np.vstack([pts_A, pts_B])
    r = (3 * kappa0 / (4 * np.pi * N)) ** (1/3)
    dist = distance_matrix(pts, pts)
    adj = (dist < r).astype(np.int8)
    np.fill_diagonal(adj, 0)
    return adj

# -----
# Un run MCMC jusqu'à brisure de symétrie
# -----

def run_until_break(N, kappa0=KAPPA0, beta=BETA, tau=TAU,
                    sym_thresh=SYM_THRESH, max_steps=5000, seed=0):
    rng = np.random.default_rng(seed)
    adj = build_mirror(N, kappa0, seed=seed)
    G = SparseBlob(N, adj)

    check_every = max(1, N // 50)    # vérifier la symétrie tous les N/50 pas

    for step in range(max_steps):

        # Vérification symétrie
        if step % check_every == 0:
            sym = G.sym_deviation()
            if sym > sym_thresh:
                return step    #  $\tau_{\text{sym}}$ 

        # Proposition MCMC
        if not G.edges:
            break
        e_rm = G.random_edge(rng)
        ne = G.random_non_edge(rng)
        if ne is None:
            continue
        u_rm, v_rm = e_rm
        u_add, v_add = ne

        #  $\Delta$  triangles
        dt_rm = G.delta_tri_remove(u_rm, v_rm)
        G._remove(u_rm, v_rm)

        # Vérifier connexité

```

```

        if not G.is_still_connected(u_rm, v_rm):
            # Remettre l'arête
            G._add(u_rm, v_rm)
            continue

        dt_add = G.delta_tri_add(u_add, v_add)
        G._add(u_add, v_add)

        delta_n = dt_add - dt_rm
        old_S = np.log(1 + G.n_tri - delta_n) - beta * G.D()
        new_S = G.Ssim(beta)
        delta_S = N * (new_S - old_S) / tau

        if delta_S < 0 and rng.random() >= np.exp(np.clip(delta_S, -500, 0)):
            # Rejeter : défaire
            G._remove(u_add, v_add)
            G._add(u_rm, v_rm)

    return max_steps # censuré

# -----
# Checkpoint I/O
# -----

def load_checkpoint():
    try:
        with open(CHECKPOINT) as f:
            return json.load(f)
    except Exception:
        return {}

def save_checkpoint(results):
    with open(CHECKPOINT, 'w') as f:
        json.dump(results, f, indent=2)

# -----
# Analyse et fit
# -----

def analyze(results):
    N_done = sorted(int(k) for k in results)
    if len(N_done) < 2:
        return

    taus = [np.median(results[str(N)]['taus']) for N in N_done]
    log_N = np.log(N_done)
    log_tau = np.log(np.maximum(taus, 1))
    coeffs = np.polyfit(log_N, log_tau, 1)

```

```

alpha, A = coeffs[0], np.exp(coeffs[1])

print(f"\n{'-'*60}")
print(f"ANALYSE CUMULATIVE - {len(N_done)} tailles")
print(f"  Loi :  $\tau_{\text{sym}} \propto N^{\{\text{alpha:.3f}\}}$  (A = {A:.3f})")
print(f"    {'N':>6} | {' $\tau$  médiane':>10} | {'Censuré':>8} | {'Ratio×Rand':>10}")
print(f"    {'-'*45}")
for N in N_done:
    r = results[str(N)]
    tau_m = np.median(r['taus'])
    cens = sum(1 for t in r['taus'] if t >= r['max_steps'])
    ratio = r.get('ratio', '?')
    print(f"    {N:>6} | {tau_m:>10.0f} | {cens:>3}/{len(r['taus'])} | {rati

if alpha > 0:
    for target in [5000, 10000, 50000]:
        N_star = (target / A) ** (1/alpha)
        print(f"    N* ( $\tau$ = {target:6d} pas)  $\approx$  {N_star:8.0f} nœuds")
print(f"{'-'*60}")

# -----
# Main
# -----

if __name__ == "__main__":
    palier = int(sys.argv[1]) if len(sys.argv) > 1 else 1
    N_list, max_steps_map = PALIERS[palier]

    results = load_checkpoint()
    print(f"{'='*60}")
    print(f"ER-Blob Scaling - Palier {palier} - N = {N_list}")
    print(f"  Seeds={N_SEEDS}, seuil brisure={SYM_THRESH}")
    print(f"  Checkpoint : {len(results)} tailles déjà calculées")
    print(f"{'='*60}")

    for N in N_list:
        key = str(N)
        if key in results:
            print(f"    N={N:5d} → déjà calculé (skip)")
            continue

        max_steps = max_steps_map[N]
        print(f"\n  N={N:5d} | max_steps={max_steps} | {N_SEEDS} seeds")
        t0 = time.time()
        taus = []
        for seed in range(N_SEEDS):
            tau = run_until_break(N, kappa0=KAPPA0, beta=BETA, tau=TAU,

```

```

        sym_thresh=SYM_THRESH,
        max_steps=max_steps,
        seed=seed * 1000 + N)

    taus.append(int(tau))
    cens = ">MAX" if tau >= max_steps else f"{tau:6d}"
    print(f"    seed {seed:2d} → τ={cens}", flush=True)

elapsed = time.time() - t0
tau_med = float(np.median(taus))
tau_min = float(np.min(taus))
tau_max = float(np.max(taus))
n_cens = sum(1 for t in taus if t >= max_steps)

# Contrôle aléatoire (3 seeds)
taus_rand = []
for seed in range(3):
    rng_c = np.random.default_rng(seed * 777 + N + 5000)
    N_half = N // 2
    pts = rng_c.uniform(0, 1, (N, 3))
    r = (3 * KAPPA0 / (4 * np.pi * N)) ** (1/3)
    d = distance_matrix(pts, pts)
    adj_r = (d < r).astype(np.int8)
    np.fill_diagonal(adj_r, 0)
    G_r = SparseBlob(N, adj_r)
    check_ev = max(1, N // 50)
    tau_r = max_steps
    for step in range(max_steps):
        if step % check_ev == 0:
            if G_r.sym_deviation() > SYM_THRESH:
                tau_r = step
                break
        if not G_r.edges: break
        e_rm = G_r.random_edge(rng_c)
        ne = G_r.random_non_edge(rng_c)
        if ne is None: continue
        u_rm, v_rm = e_rm
        u_add, v_add = ne
        G_r._remove(u_rm, v_rm)
        if not G_r.is_still_connected(u_rm, v_rm):
            G_r._add(u_rm, v_rm); continue
        G_r._add(u_add, v_add)
    taus_rand.append(int(tau_r))

tau_rand_med = float(np.median(taus_rand))
ratio = f"{tau_med/max(tau_rand_med,1):.1f}×" if tau_rand_med > 0 else "∞"

results[key] = {

```

```

        'N':          N,
        'taus':      taus,
        'tau_med':   tau_med,
        'tau_min':   tau_min,
        'tau_max':   tau_max,
        'n_censored': n_cens,
        'tau_rand':  tau_rand_med,
        'ratio':     ratio,
        'max_steps': max_steps,
        'elapsed_s': round(elapsed, 1),
    }
    save_checkpoint(results)

    print(f" → τ médiane={tau_med:.0f}  rand={tau_rand_med:.0f}  "
          f"ratio={ratio}  censuré={n_cens}/{N_SEEDS}  ({elapsed:.0f}s)")

    analyze(results)
    print(f"\nCheckpoint sauvegardé : {CHECKPOINT}")

# — Mode single-N —————
def run_single(N, n_seeds, max_steps):
    """Lance un seul N avec checkpoint. Appelé depuis la ligne de commande."""
    results = load_checkpoint()
    key = str(N)
    if key in results:
        print(f"N={N} déjà calculé — skip.")
        analyze(results)
        return

    print(f"\n{' '*60}")
    print(f"ER-Blob Single — N={N} | seeds={n_seeds} | max_steps={max_steps}")
    print(f"{' '*60}")
    t0 = time.time()
    taus = []
    for seed in range(n_seeds):
        tau = run_until_break(N, kappa0=KAPPA0, beta=BETA, tau=TAU,
                              sym_thresh=SYM_THRESH,
                              max_steps=max_steps, seed=seed*1000+N)
        taus.append(int(tau))
        cens = ">MAX" if tau >= max_steps else f"{tau:6d}"
        print(f"  seed {seed:2d} → τ={cens}", flush=True)

    elapsed = time.time() - t0
    tau_med = float(np.median(taus))
    n_cens = sum(1 for t in taus if t >= max_steps)

    results[key] = {

```

```

        'N': N, 'taus': taus,
        'tau_med': tau_med,
        'tau_min': float(np.min(taus)),
        'tau_max': float(np.max(taus)),
        'n_censored': n_cens,
        'tau_rand': 0.0, 'ratio': '∞',
        'max_steps': max_steps,
        'elapsed_s': round(elapsed, 1),
    }
    save_checkpoint(results)
    print(f"\n → τ médiane={tau_med:.0f}   censuré={n_cens}/{n_seeds}   ({elapsed:
analyze(results)

```